Name: Sri hari siddharth M
Roll No: 241201055
Year & Dept: 3 & MCT

Experiment 1: Case Study on the Analysis of SDLC Models Applied to a Real-World Software Application

1. Aim

To study the Software Development Life Cycle (SDLC) models and analyse which model best fits the development and maintenance of a real-world software application, namely an online railway ticket reservation system of the kind operated by Indian Railways through IRCTC.

2. Introduction to SDLC

The Software Development Life Cycle is a structured sequence of phases used to build and maintain software: requirement analysis, system design, implementation (coding), testing, deployment and maintenance. A process model defines how these phases are ordered and how often they are repeated. The choice of model affects cost, schedule, quality, risk control and the ability to accept changes after the project has started.

The main models studied in this course are the Waterfall model, the Iterative and Incremental model, the Spiral model, the V-Model and Agile methodologies such as Scrum and Kanban. Waterfall moves in one forward direction and assumes stable requirements. Iterative and Incremental models deliver working software in cycles. The Spiral model adds explicit risk analysis in each loop. The V-Model pairs every development phase with a testing phase. Agile models work in short time-boxed sprints with continuous customer feedback.

3. Description of the Case Study Application

An online ticket reservation system must serve millions of users, handle concurrent seat booking without double-selling a seat, integrate payment gateways, support multiple languages and devices, and meet strict security and availability requirements during peak festival seasons. Requirements are partly stable, such as train schedules, fare structures and the basic booking workflow. Other requirements are volatile, such as new payment methods, mobile features, waiting-list rules, regulatory changes and promotional schemes. Because of this mixture, the system evolves continuously after its first release.

The main modules identified for the analysis are: user registration and login, train search and availability, seat selection and booking, payment processing, ticket cancellation and refund, notifications by SMS and email, and an administrative reporting module.

4. Comparison of SDLC Models for This Application

SDLC Model	Suitability	Reason
Waterfall	Low	Requirements change after launch; late testing risks costly defects in booking and payment logic.
Iterative and Incremental	High	Modules such as search, booking and payment can be released in increments and refined.

SDLC Model	Suitability	Reason
Spiral	Medium to High	Strong risk analysis for payments and security, but heavier documentation and management overhead.
V-Model	Medium	Good test traceability for critical modules, but less adaptive to changing requirements.
Agile (Scrum)	High	Sprints, frequent stakeholder feedback and continuous integration suit a system that evolves continuously.

5. Analysis

The application is best served by a hybrid approach. Core, stable parts such as payment security, compliance and seat-locking logic can follow a Spiral or V-Model style with formal risk reviews, threat modelling and traceable test cases. Customer-facing features such as search, seat maps and notifications are better served by Agile sprints with short feedback loops and A/B testing of interface changes.

The workflow for a single change, such as adding a new payment method, illustrates the difference. In a Waterfall process the change would wait for the next major release cycle and go through the full documentation chain. In an Agile and DevOps process it would be planned in a sprint, built behind a feature flag, tested automatically in a pipeline and released to a small group of users before full rollout. The second approach reduces the cost of change and the time to benefit.

6. Observations

- Requirement volatility is the main factor that rules out a pure Waterfall approach for the customer-facing system.
- Payment and security modules justify risk-driven activities such as threat modelling, penetration testing and formal review.
- Seat booking requires concurrency control; this is a technical risk that should be prototyped early in an iterative cycle.
- Frequent releases combined with monitoring and rollback capability reduce the cost of late defect discovery.
- Large public platforms combine an iterative model with DevOps automation, so that testing and deployment are repeatable.

7. Limitations of the Study

This study is based on publicly visible features and general industry practice. It does not use internal design documents or project records of the operator. The suitability ratings are qualitative judgements and would need to be validated with project data such as defect rates, release frequency and customer satisfaction scores.

8. Result and Conclusion

The study concludes that an iterative and incremental, Agile-based approach with risk-driven reviews for critical modules is the most suitable SDLC strategy for an online railway reservation system. The choice of model should follow three factors: the stability of requirements, the criticality of each module and the expected pace of change. A single model rarely fits every part of a large system, so mature organisations combine models according to the needs of each component.

9. References

- Pressman, R. S., and Maxim, B. R., Software Engineering: A Practitioner's Approach, McGraw-Hill Education.
- Sommerville, I., Software Engineering, Pearson Education.